

Experiment No.	1
Experiment Title	Working with Python Packages – NumPy, SciPy, Scikit-Learn, Matplotlib ¹
Student Name	T.Ashwath Ram
Register Number	3122247001007
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	25/07/2026

1 Objective

The objective of this experiment is to explore the core Python libraries used in a machine learning workflow – NumPy, Pandas, SciPy, Scikit-learn and Matplotlib/Seaborn – and to apply them on five publicly available datasets (Iris, Loan Approval, Diabetes Prediction, Email Spam and MNIST) in order to understand array manipulation, data preprocessing, exploratory data analysis, feature selection, model building and performance evaluation. A further objective is to identify, for each dataset, the correct category of machine learning task (supervised / unsupervised, classification / regression) and the most suitable algorithm and feature selection technique for it.

2 Problem Statement

Five datasets, each representing a different real-world prediction problem, were obtained from public repositories (Kaggle / UCI). For every dataset the following pipeline was implemented: loading the data, performing exploratory data analysis and visualization, preprocessing (handling missing values, encoding, scaling), feature selection, splitting the data into training and testing sets, training one or more Scikit-learn classifiers, and evaluating the models using accuracy, confusion matrix and classification-report metrics. The goal is not a single input-output mapping but a comparative understanding of how the ML workflow changes across tabular numeric data (Diabetes, Loan), tabular mixed data (Loan), text data (Spam) and image data (MNIST).

3 Python Library Exploration

The table below summarizes the key functions/classes from each library that were used across the five notebooks.

Library	Key functions/classes used
NumPy	Array reshaping (<code>values.reshape(28,28)</code> for MNIST image display), vectorised numeric operations underlying Pandas/Scikit-learn computations.
Pandas	<code>read_csv</code> , <code>head</code> , <code>tail</code> , <code>info</code> , <code>describe</code> , <code>isnull().sum()</code> , <code>duplicated()</code> , <code>drop_duplicates()</code> , <code>drop</code> , <code>value_counts</code> , <code>corr</code> , <code>columns.str.strip()</code> , <code>hist</code> , <code>boxplot</code> .
SciPy / Statistics	Statistical tests underlying <code>f_classif</code> (ANOVA F-test) and <code>chi2</code> used inside Scikit-learn's feature-selection module.

Scikit-learn	<code>train_test_split</code> , <code>StandardScaler</code> , <code>LabelEncoder</code> , <code>TfidfVectorizer</code> , <code>SelectKBest</code> (with <code>f_classif/chi2</code>), <code>DecisionTreeClassifier</code> , <code>LogisticRegression</code> , <code>KNeighborsClassifier</code> , <code>accuracy_score</code> , <code>confusion_matrix</code> , <code>classification_report</code> .
Matplotlib / Seaborn	<code>plt.hist/df.hist</code> , <code>plt.scatter</code> , <code>plt.boxplot</code> / <code>df.boxplot</code> , <code>plt.imshow</code> , <code>sns.heatmap</code> , <code>sns.pairplot</code> , <code>sns.countplot</code> .

4 Dataset-wise Machine Learning Task Identification

The table below (as required by the experiment) summarizes the type of ML task, the feature selection technique explored, and the algorithm(s) applied for each dataset.

Dataset	Type of ML Task	Feature Selection Technique	Algorithm(s) Applied
Iris Dataset	Supervised – Multiclass Classification	SelectKBest with Chi-square (<code>chi2</code>)	Decision Tree, K-Nearest Neighbours (<code>k=3</code>)
Loan Amount Prediction	Supervised – Binary Classification (loan approval status)	SelectKBest with ANOVA F-test (<code>f_classif</code>)	Decision Tree, Logistic Regression
Predicting Diabetes	Supervised – Binary Classification	SelectKBest with ANOVA F-test (<code>f_classif</code>)	Decision Tree, Logistic Regression
Classification of Email Spam	Supervised – Binary Text Classification (NLP)	TF-IDF vectorisation + SelectKBest with ANOVA F-test	Decision Tree, Logistic Regression
Handwritten Digit Recognition (MNIST)	Supervised – Multiclass Image Classification	SelectKBest with Chi-square (<code>chi2</code>), $k = 300$ pixels	Decision Tree

5 Dataset 1: Iris Dataset

5.1 Dataset Description

Dataset Name	Iris Dataset
Dataset Source	Kaggle / UCI Machine Learning Repository (<code>Iris.csv</code>)
Number of Samples	150

Number of Features	4 (SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm); the Id column was dropped as it carries no predictive value
Number of Classes	3 (Iris-setosa, Iris-versicolor, Iris-virginica)
Missing Values	None (0 across all columns)
Train-Test Split	80:20 (120 train / 30 test samples)

5.2 Exploratory Data Analysis

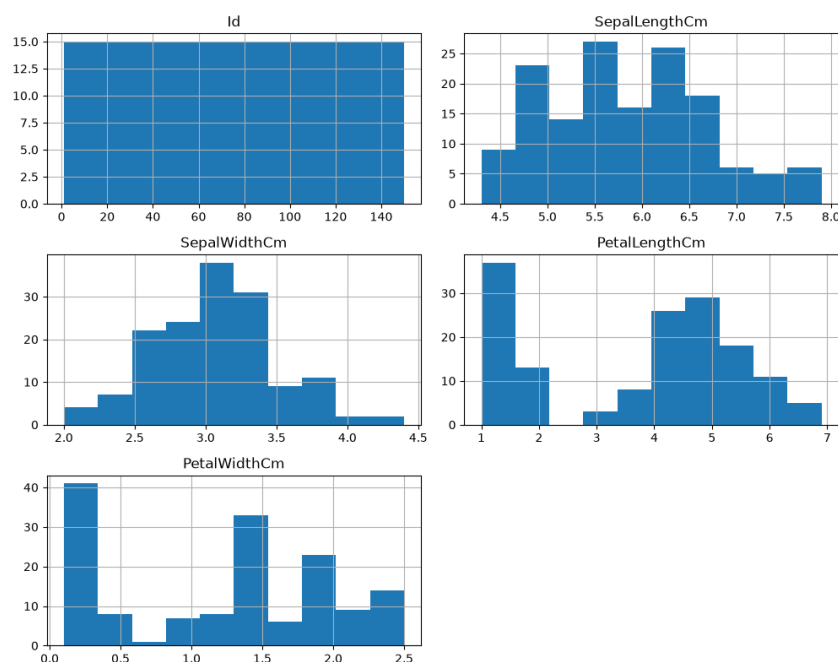


Figure 1: Histogram of the four numeric features of the Iris dataset

Inference: All four features are unimodal, with *PetalLengthCm* and *PetalWidthCm* showing a clearly bimodal-looking spread caused by the separation between *setosa* and the other two species. *SepalWidthCm* is close to a normal distribution. This already hints that petal measurements will be more discriminative than sepal measurements for classification.

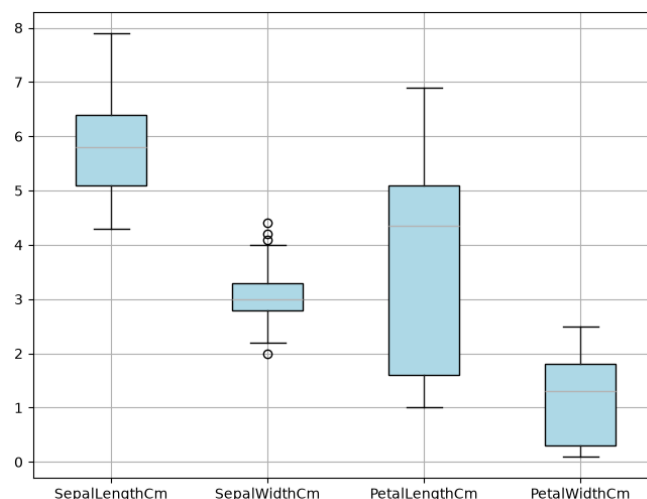


Figure 2: Boxplot of Iris features (excluding Id)

Inference: SepalWidthCm has a few mild outliers beyond the whiskers, while PetalLengthCm and PetalWidthCm show a wide spread with no extreme outliers. The scales of petal measurements are larger and more variable than sepal measurements.

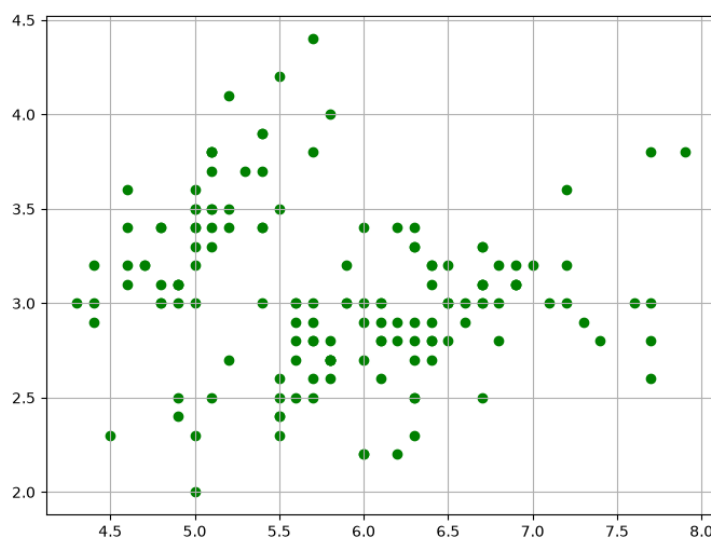


Figure 3: Scatter plot of SepalLengthCm vs SepalWidthCm

Inference: Sepal length and width alone do not separate the three classes cleanly – the points overlap considerably, reinforcing why petal-based features (seen next) are more useful for classification.

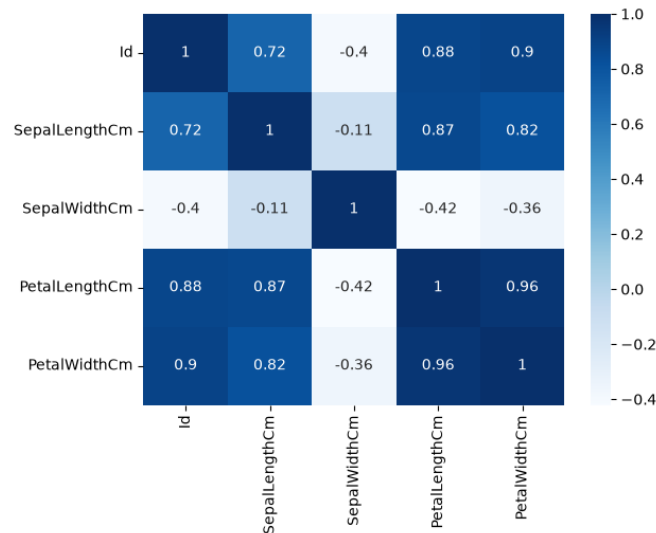


Figure 4: Correlation heatmap of Iris features

Inference: PetalLengthCm and PetalWidthCm are very strongly correlated with each other ($r \approx 0.96$) and both are strongly correlated with SepalLengthCm, while SepalWidthCm is weakly/negatively correlated with the rest. This multicollinearity is expected since petal size grows proportionally.

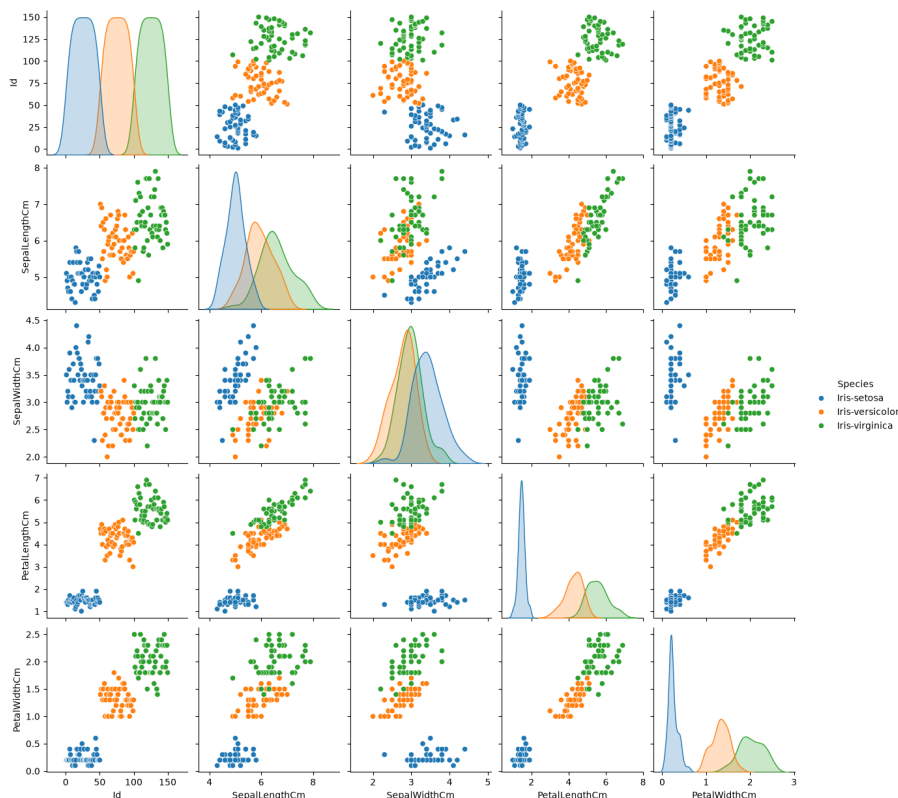


Figure 5: Pairplot of all features coloured by Species

Inference: The pairplot confirms that *Iris-setosa* is linearly separable from the other two species on almost every feature pair, while *versicolor* and *virginica* overlap slightly,

mainly on sepal measurements – explaining why simple models still reach very high accuracy on this dataset.

5.3 Data Preprocessing

- **Missing values:** None found; no imputation required.
- **Irrelevant feature removal:** The Id column was dropped.
- **Encoding:** The categorical target **Species** was label-encoded (`LabelEncoder`) into $\{0, 1, 2\}$.
- **Feature scaling:** Not required for tree-based/KNN models on this small, similarly scaled feature set (not applied in this notebook).
- **Train-test split:** 80:20 split using `train_test_split(random_state=42)`.

```

1 from sklearn.preprocessing import LabelEncoder
2 le = LabelEncoder()
3 df["Species"] = le.fit_transform(df["Species"])
4
5 from sklearn.model_selection import train_test_split
6 x = df.drop("Species", axis=1)
7 y = df["Species"]
8 x_train, x_test, y_train, y_test = train_test_split(
9     x, y, test_size=0.2, random_state=42)

```

5.4 Feature Selection

`SelectKBest` with the Chi-square (`chi2`) score function was used ($k = 1$). The scores obtained were: `SepalLengthCm` = 8.23, `SepalWidthCm` = 3.12, `PetalLengthCm` = 92.08, `PetalWidthCm` = 52.60 – confirming **PetalLengthCm** as the single most informative feature, consistent with the EDA observations above.

5.5 Results

Metric	Decision Tree	K-Nearest Neighbours (k=3)
Accuracy	1.00	1.00
Precision (macro avg)	1.00	–
Recall (macro avg)	1.00	–
F1-score (macro avg)	1.00	–

Confusion Matrix (Decision Tree): $\begin{bmatrix} 10 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 11 \end{bmatrix}$

Inference: Both models achieve perfect classification on the held-out test set. This is expected given how well-separated the Iris classes are, as seen in the pairplot; it also suggests the test split is small (30 samples) and easy, so results should be interpreted with that caveat rather than as evidence the problem is trivial in general.

6 Dataset 2: Loan Amount Prediction

6.1 Dataset Description

Dataset Name	Loan Approval Dataset
Dataset Source	Kaggle (<code>loan_approval_dataset.csv</code>)
Number of Samples	4,269
Number of Features	12 predictors (<code>no_of_dependents</code> , <code>education</code> , <code>self_employed</code> , <code>income_annum</code> , <code>loan_amount</code> , <code>loan_term</code> , <code>cibil_score</code> , <code>residential_assets_value</code> , <code>commercial_assets_value</code> , <code>luxury_assets_value</code> , <code>bank_asset_value</code>) plus <code>loan_id</code>
Number of Classes	2 (Approved / Rejected)
Missing Values	None (0 across all columns)
Train-Test Split	80:20 (3,415 train / 854 test samples)

6.2 Exploratory Data Analysis

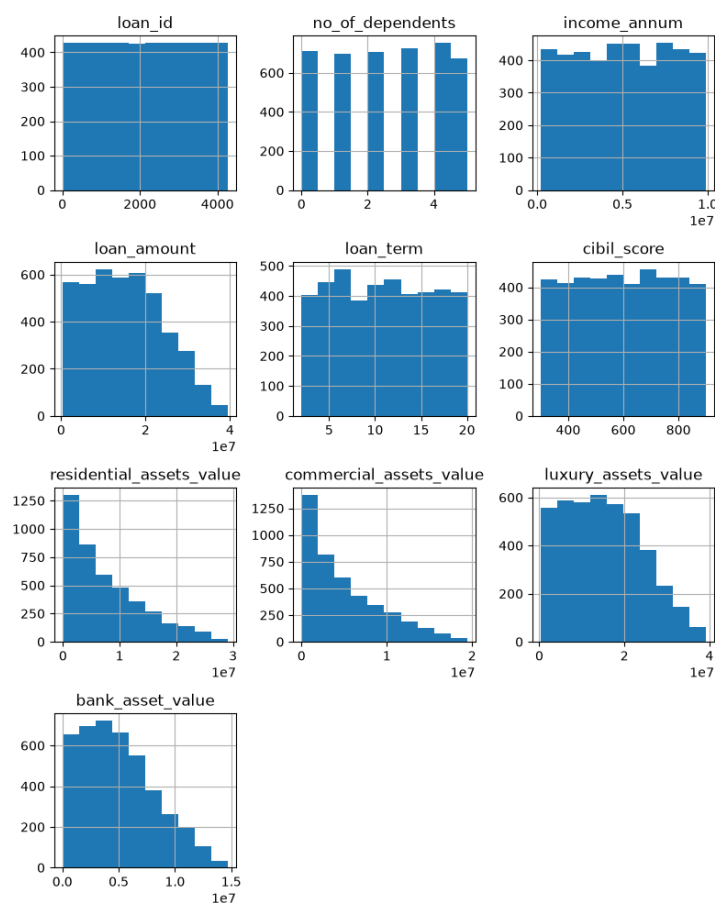


Figure 6: Histograms of all numeric loan features

Inference: Income, loan amount and asset-value columns are fairly uniformly spread across a wide range rather than normally distributed, while `cibil_score` spans the full 300–900 range roughly uniformly. This wide, roughly uniform spread suggests the dataset was likely synthetically generated.

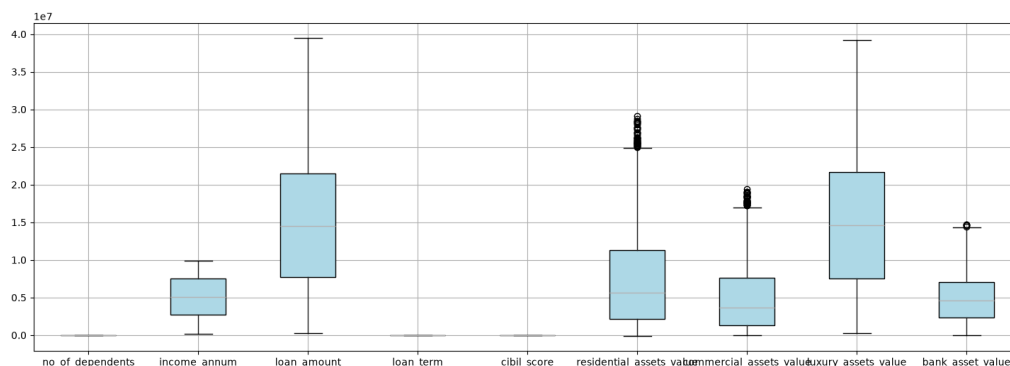


Figure 7: Boxplot of all numeric loan features

Inference: Asset value columns (`residential`/`commercial`/`luxury`/`bank`) show some right-skew with a handful of high-value outliers, while `cibil_score` and `loan_term` are tightly bounded with almost no outliers.

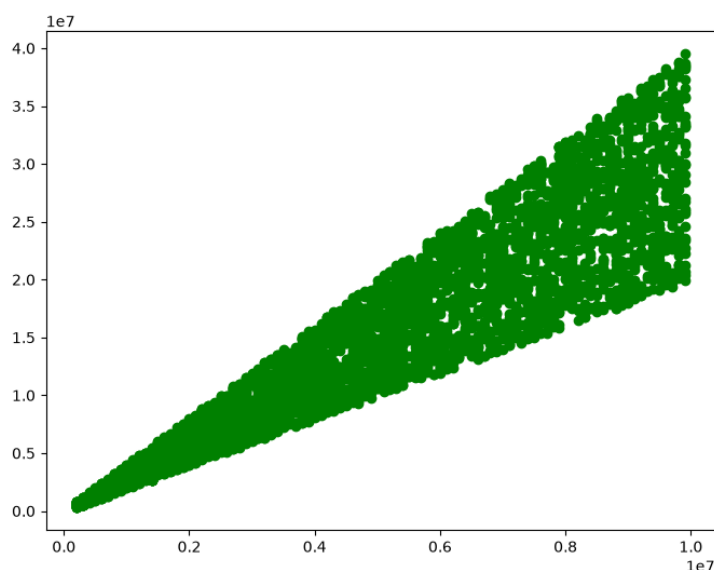


Figure 8: Scatter plot of Annual Income vs Loan Amount

Inference: There is a clear positive linear trend between annual income and loan amount requested, which is intuitive – applicants with higher income tend to request larger loans.

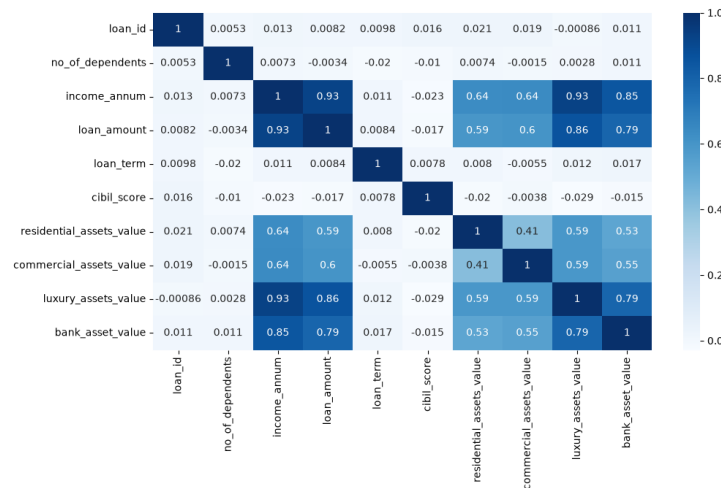


Figure 9: Correlation heatmap of numeric loan features

Inference: `loan_amount` correlates strongly with `income_annum` and with the asset-value columns, while `cibil_score` shows very weak linear correlation with the other numeric features – yet, as feature selection shows below, it is the single strongest predictor of approval, meaning the relationship with the target is not simply linear covariance with other inputs.

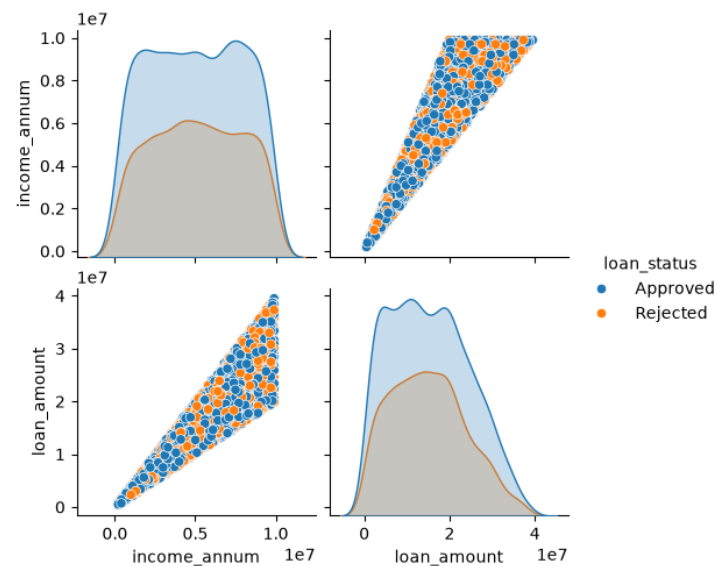


Figure 10: Pairplot of Income, Loan Amount coloured by Loan Status

Inference: Approved and rejected applicants overlap heavily on income and loan-amount alone, indicating that these two features by themselves are not sufficient to separate the classes – consistent with `cibil_score` (not shown here) being the dominant discriminating factor.

6.3 Data Preprocessing

- **Column name cleanup:** Column names had leading whitespace (e.g. " education") which was stripped using `df.columns.str.strip()`.

- **Missing values / duplicates:** None found.
- **Encoding:** education, self_employed and loan_status were label-encoded.
- **Train-test split:** 80:20 split with random_state=42.

```

1 df.columns = df.columns.str.strip()
2
3 from sklearn.preprocessing import LabelEncoder
4 le_edu = LabelEncoder(); le_self = LabelEncoder(); le_status =
  LabelEncoder()
5 df["education"] = le_edu.fit_transform(df["education"])
6 df["self_employed"] = le_self.fit_transform(df["self_employed"])
7 df["loan_status"] = le_status.fit_transform(df["loan_status"])

```

6.4 Feature Selection

SelectKBest with the ANOVA F-test (`f_classif`) was used. `cibil_score` obtained by far the highest F-score (≈ 5018), dwarfing every other feature – confirming it as the dominant predictor of loan approval status.

6.5 Results

Metric	Decision Tree	Logistic Regression
Accuracy	0.974	0.822
Precision (class 1)	–	0.83
Recall (class 1)	–	0.66
F1-score (class 1)	–	0.73

Confusion Matrix (Logistic Regression): $\begin{bmatrix} 492 & 44 \\ 108 & 210 \end{bmatrix}$

Inference: The Decision Tree clearly outperforms Logistic Regression here, suggesting the true decision boundary (largely driven by CIBIL score thresholds) is closer to a set of rules/splits than to a linear combination of features. Logistic Regression underpredicts the minority "rejected" class more often (108 false negatives), indicating room for improvement via feature scaling or class-balancing.

7 Dataset 3: Predicting Diabetes

7.1 Dataset Description

Dataset Name	Diabetes Prediction Dataset
Dataset Source	Kaggle (<code>Diabetes_prediction.csv</code>)
Number of Samples	1,000
Number of Features	8 (Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age)

Number of Classes	2 (Diagnosis: 0 = non-diabetic, 1 = diabetic)
Missing Values	None (0 across all columns)
Train-Test Split	80:20 (800 train / 200 test samples)

7.2 Exploratory Data Analysis

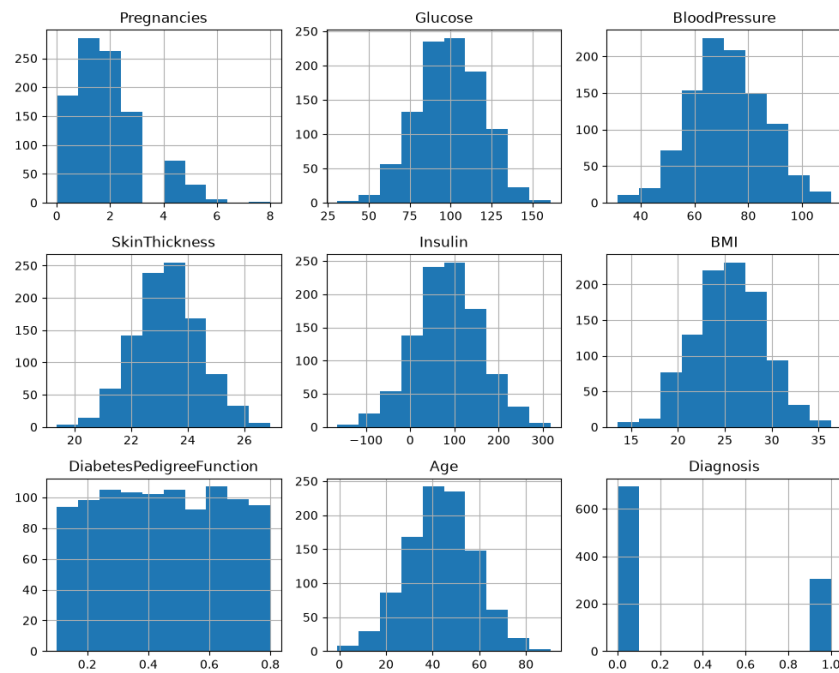


Figure 11: Histograms of all Diabetes dataset features

Inference: Glucose, BloodPressure and Age approximate a normal distribution, while Diagnosis shows clear class imbalance (about 30% positive, 70% negative – mean ≈ 0.306), which needs to be kept in mind when evaluating model performance.

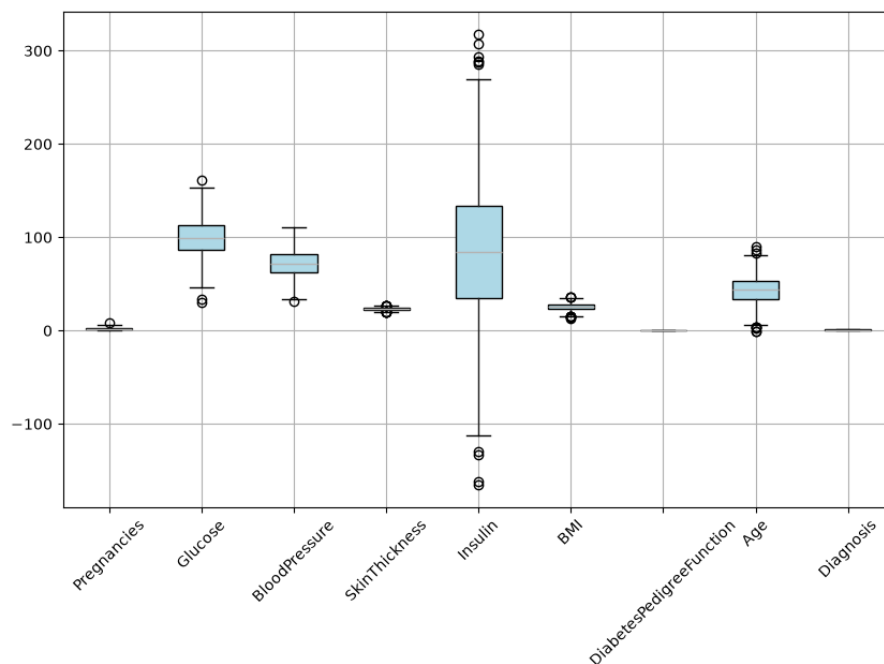


Figure 12: Boxplot of all Diabetes dataset features

Inference: `Insulin` shows the widest spread and even negative values ($\min \approx -165$), which is not physiologically valid and points to synthetic/noisy data rather than a data-entry error to be cleaned in the usual sense.

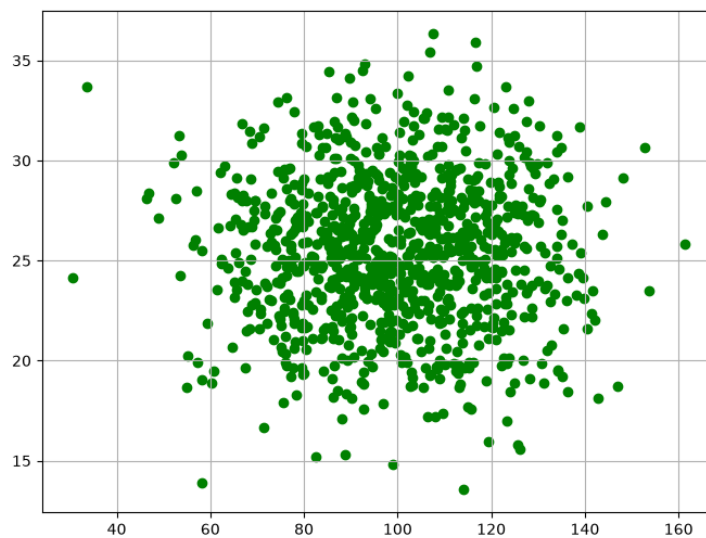


Figure 13: Scatter plot of Glucose vs BMI

Inference: No strong linear relationship is visible between Glucose and BMI in this scatter plot, suggesting these two features contribute independent information to the model.

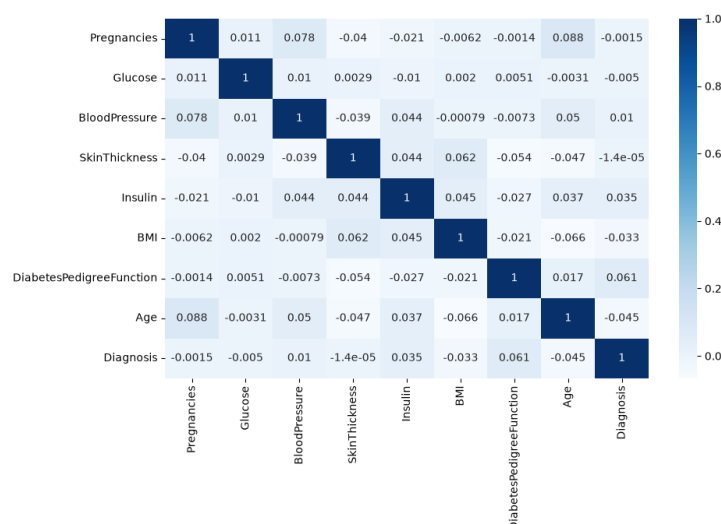


Figure 14: Correlation heatmap of Diabetes features

Inference: None of the individual features shows a strong linear correlation with **Diagnosis** (all correlations are weak), which foreshadows the relatively poor accuracy of the linear model compared to what one might expect from a medical diagnosis dataset.

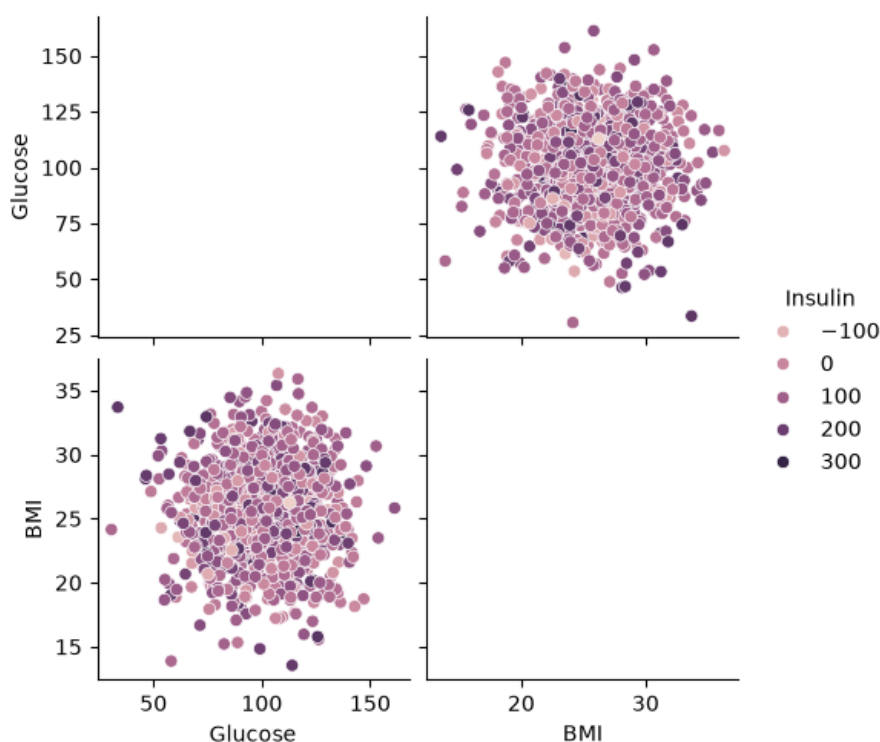


Figure 15: Pairplot of Glucose, BMI and Insulin

Inference: The classes overlap substantially across these three features with no clean separating boundary, which is consistent with the modest classification accuracy obtained later.

7.3 Data Preprocessing

- **Missing values / duplicates:** None found; zero-value sanity checks on Glucose, BloodPressure, SkinThickness, Insulin and BMI also returned 0 (no biologically-implausible zeros).
- **Feature scaling:** `StandardScaler` was applied to standardize all features before model training, which matters for Logistic Regression.
- **Train-test split:** 80:20 split with `random_state=42`.

```

1 from sklearn.model_selection import train_test_split
2 X_train, X_test, y_train, y_test = train_test_split(
3     x, y, test_size=0.2, random_state=42)
4
5 from sklearn.preprocessing import StandardScaler
6 scaler = StandardScaler()
7 X_train = scaler.fit_transform(X_train)
8 X_test = scaler.transform(X_test)

```

7.4 Feature Selection

`SelectKBest` with `f_classif` ($k = 1$) ranked **Age** as the strongest single predictor (F-score ≈ 6.11), followed by BMI (≈ 5.06); Pregnancies and BloodPressure scored near zero.

7.5 Results

Metric	Decision Tree	Logistic Regression
Accuracy	0.53	0.685
Precision (class 1)	–	0.00
Recall (class 1)	–	0.00
F1-score (class 1)	–	0.00

Confusion Matrix (Logistic Regression): $\begin{bmatrix} 137 & 0 \\ 63 & 0 \end{bmatrix}$

Inference: Although Logistic Regression shows a higher raw accuracy (0.685) than the Decision Tree (0.53), the confusion matrix reveals that it predicts *every* sample as the majority class (0), so its precision/recall/F1 for the diabetic class are all zero – the accuracy figure is misleading here because of class imbalance. The Decision Tree, despite lower accuracy, at least attempts to predict both classes. This is an important finding: accuracy alone is not a reliable metric for this imbalanced dataset, and techniques such as class weighting, oversampling (e.g. SMOTE), or threshold tuning are needed for a fair comparison.

8 Dataset 4: Classification of Email Spam

8.1 Dataset Description

Dataset Name	SMS/Email Spam Collection
Dataset Source	Kaggle / UCI (spam.csv)
Number of Samples	5,572 raw messages; 5,169 after duplicate removal
Number of Features	1 raw text feature (v2 , the message body), expanded to 8,672 TF-IDF features after vectorisation
Number of Classes	2 (ham / spam)
Missing Values	Three unnamed columns (Unnamed: 2/3/4) were almost entirely null (5,522 / 5,560 / 5,566 missing out of 5,572) and were dropped
Train-Test Split	80:20 (4,135 train / 1,034 test messages)

8.2 Exploratory Data Analysis

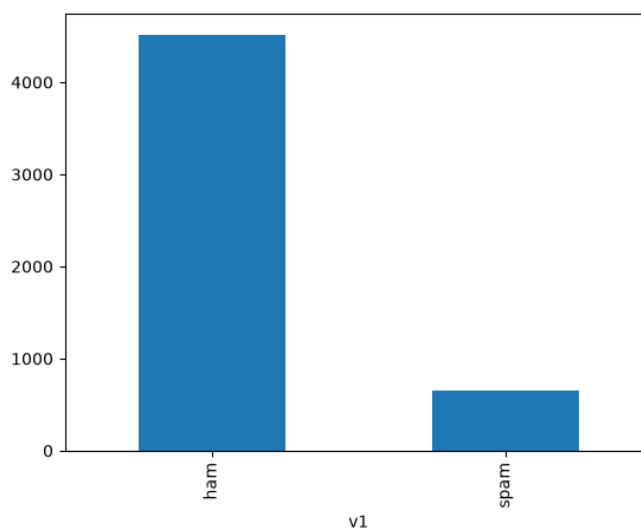


Figure 16: Class distribution of ham vs spam messages

Inference: The dataset is heavily imbalanced towards **ham** messages (originally 4,825 ham vs 747 spam, roughly 87:13), so accuracy alone can be misleading and precision/recall on the spam class matter more.

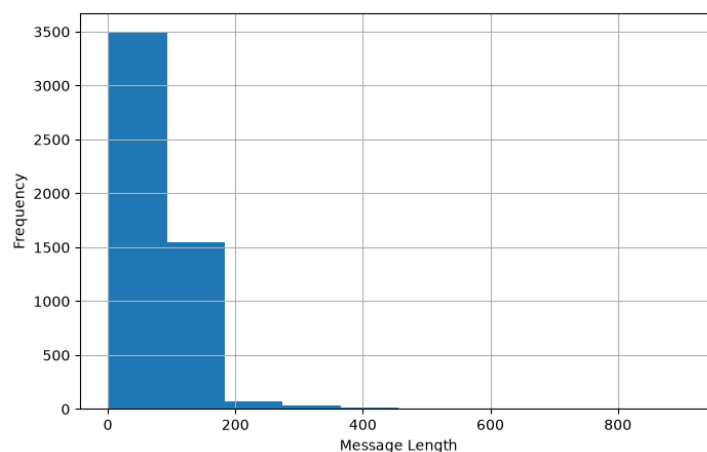


Figure 17: Distribution of message length

Inference: Message length is right-skewed, with most messages under roughly 150 characters and a long tail of longer messages – spam messages are generally known to run longer on average due to promotional text, which is explored further via the boxplot.

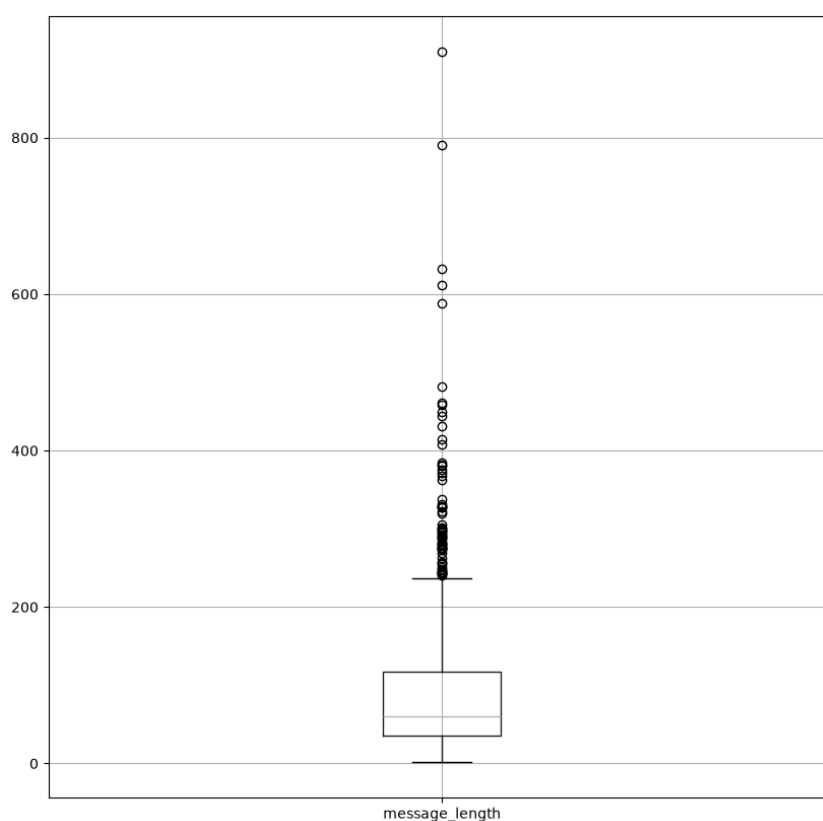


Figure 18: Boxplot of numeric columns (message length)

Inference: The boxplot of message length shows several outliers on the higher end, corresponding to unusually long messages; these were retained rather than removed since message length itself can be a useful discriminating signal between spam and ham.

8.3 Data Preprocessing

- **Missing values:** The three near-empty `Unnamed` columns were dropped.
- **Duplicates:** 403 duplicate rows were identified and removed with `drop_duplicates()`.
- **Encoding:** The target label (`v1`: ham/spam) was label-encoded.
- **Feature engineering / text vectorisation:** Raw text was converted into numeric features using `TfidfVectorizer`, producing an 8,672-dimensional sparse feature matrix.
- **Train-test split:** 80:20 split with `random_state=42`.

```

1 df = df.drop(["Unnamed: 2", "Unnamed: 3", "Unnamed: 4"], axis=1)
2 df = df.drop_duplicates()
3
4 from sklearn.preprocessing import LabelEncoder
5 from sklearn.feature_extraction.text import TfidfVectorizer
6 from sklearn.model_selection import train_test_split
7
8 tfidf = TfidfVectorizer()
9 le = LabelEncoder()
10 df["v1"] = le.fit_transform(df["v1"])
11 x = tfidf.fit_transform(df["v2"])
12 y = df["v1"]
13
14 x_train, x_test, y_train, y_test = train_test_split(
15     x, y, test_size=0.2, random_state=42)

```

8.4 Feature Selection

`SelectKBest` with `f_classif` ($k = 10$) was applied on the TF-IDF matrix. The top 10 words selected were: *150p*, *call*, *claim*, *free*, *mobile*, *prize*, *txt*, *uk*, *urgent*, *www* – all classic spam/promotional vocabulary, which validates that TF-IDF plus ANOVA scoring correctly surfaces spam-indicative terms.

8.5 Results

Metric	Decision Tree	Logistic Regression
Accuracy	0.967	0.956

Confusion Matrix (Logistic Regression): $\begin{bmatrix} 886 & 3 \\ 43 & 102 \end{bmatrix}$

Inference: Both models perform well, with the Decision Tree marginally ahead in raw accuracy. Logistic Regression makes very few false positives (only 3 ham messages misclassified as spam) but misses 43 actual spam messages (false negatives), which matters in a real deployment since a missed spam message is usually less costly than a legitimate message being blocked – so the current threshold/model choice is reasonably well suited to this asymmetry.

9 Dataset 5: Handwritten Digit Recognition (MNIST)

9.1 Dataset Description

Dataset Name	MNIST (test split)
Dataset Source	Kaggle / Yann LeCun's MNIST database (<code>mnist_test.csv</code>)
Number of Samples	10,000
Number of Features	784 (28×28 grayscale pixel intensities, columns <code>1x1 ... 28x28</code>)
Number of Classes	10 (digits 0–9)
Missing Values	None (0 across all columns)
Train-Test Split	80:20 (8,000 train / 2,000 test samples)

9.2 Exploratory Data Analysis

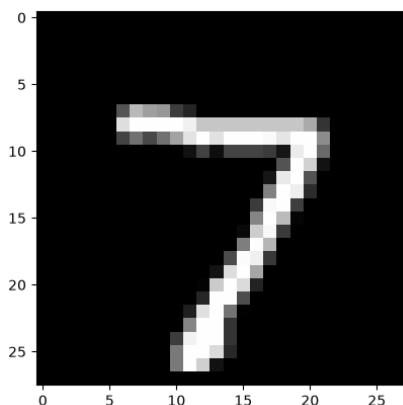


Figure 19: Sample MNIST digit reshaped to 28×28 and displayed with `imshow`

Inference: Visualising a sample confirms the data was loaded and reshaped correctly from the flattened 784-length pixel vector back into a recognisable digit image, which is a necessary sanity check before any modelling on image data of this kind.

Class distribution (from `value_counts()`): label 1: 1,135, label 2: 1,032, label 7: 1,028, label 3: 1,010, label 9: 1,009, label 4: 982, label 0: 980, label 8: 974, label 6: 958, label 5: 892 – the classes are reasonably well balanced, with digit "1" being the most frequent and digit "5" the least frequent.

9.3 Data Preprocessing

- **Missing values:** None found across all 785 columns.
- **Feature/target split:** The `label` column was separated as the target; the remaining 784 pixel columns form the feature matrix.
- **Scaling:** Not applied in this notebook (pixel values were used as-is, 0–255); normalising pixel values to `[0, 1]` is a common improvement that could be added.

- **Train-test split:** 80:20 split with `random_state=42`.

```

1 x = df.drop("label", axis=1)
2 y = df["label"]
3
4 from sklearn.model_selection import train_test_split
5 x_train, x_test, y_train, y_test = train_test_split(
6     x, y, test_size=0.2, random_state=42)

```

9.4 Feature Selection

Given the high dimensionality (784 pixel features), `SelectKBest` with the Chi-square (`chi2`) score function was used to reduce the feature set to the $k = 300$ most informative pixels (typically the central pixels where digit strokes are most likely to appear, since border pixels are almost always zero/blank).

9.5 Results

Metric	Decision Tree (784 features)	Decision Tree (300 selected features)
Accuracy	0.794	0.793

Inference: Reducing the feature space from 784 to the 300 most informative pixels via Chi-square selection retains essentially the same accuracy (0.793 vs 0.794), confirming that a large fraction of the 784 pixel columns (mostly border pixels that are always blank) carry very little discriminative information. This is a useful result for efficiency – a lighter model can be trained with almost no loss in accuracy. The per-class confusion matrix shows the Decision Tree confuses visually similar digit pairs most often (e.g., 4 vs 9, 3 vs 5), which is consistent with how these digits are drawn.

10 Machine Learning Algorithms – Theory

10.1 Decision Tree Classifier

Working principle: A Decision Tree recursively splits the feature space into regions by choosing, at each node, the feature and threshold that most reduces impurity (commonly Gini impurity or entropy) in the resulting child nodes.

Mathematical formulation: For a node t with class proportions p_i , the Gini impurity is

$$G(t) = 1 - \sum_{i=1}^C p_i^2,$$

and the split that maximises the impurity decrease $\Delta G = G(t) - \frac{N_L}{N}G(t_L) - \frac{N_R}{N}G(t_R)$ is chosen at each node.

Advantages: Easy to interpret, handles both numeric and categorical features, requires no feature scaling, captures non-linear decision boundaries (as seen in the Loan and Diabetes results above, where it outperformed Logistic Regression).

Limitations: Prone to overfitting on noisy data with default (unbounded) depth, can be unstable to small changes in data, and does not naturally exploit linear relationships as efficiently as linear models.

10.2 Logistic Regression

Working principle: Logistic Regression models the log-odds of the positive class as a linear combination of the input features, mapped to a probability through the sigmoid function.

Mathematical formulation:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}},$$

with parameters learned by minimising the binary cross-entropy (log-loss).

Advantages: Fast to train, interpretable coefficients, works well when classes are linearly separable and benefits from feature scaling (as applied for Diabetes).

Limitations: Assumes a linear decision boundary in feature space, is sensitive to class imbalance (as seen clearly in the Diabetes results, where it collapsed to predicting only the majority class), and needs explicit regularisation/class-weighting to handle skewed data.

10.3 K-Nearest Neighbours (KNN)

Working principle: KNN is a non-parametric, instance-based method that classifies a new point by majority vote among its k nearest neighbours in feature space, typically using Euclidean distance.

Mathematical formulation: For a query point $\mathbf{x}$,

$$\hat{y} = \arg \max_c \sum_{i \in N_k(\mathbf{x})} \mathbb{I}(y_i = c),$$

where $N_k(\mathbf{x})$ is the set of the k closest training points to $\mathbf{x}$.

Advantages: Simple, makes no assumption about the underlying data distribution, and performed extremely well on the well-separated Iris dataset.

Limitations: Computationally expensive at prediction time for large datasets, sensitive to feature scaling and to the choice of k , and degrades in high-dimensional or noisy feature spaces.

10.4 TF-IDF with Decision Tree / Logistic Regression (Text Classification)

Working principle: Raw text is converted into a numeric matrix using Term Frequency–Inverse Document Frequency (TF-IDF) weighting, after which a standard classifier (Decision Tree or Logistic Regression) is trained on the resulting sparse feature matrix.

Mathematical formulation: For term t in document d ,

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \log\left(\frac{N}{\text{df}(t)}\right),$$

where $\text{tf}(t, d)$ is the term frequency, N is the total number of documents and $\text{df}(t)$ is the number of documents containing term t .

Advantages: Down-weights common, uninformative words (e.g. "the", "is") and up-weights rare, discriminative words (e.g. "prize", "urgent"), which is why the ANOVA-selected top words for spam were clearly promotional in nature.

Limitations: Produces very high-dimensional, sparse feature vectors (8,672 features here), ignores word order and context, and cannot capture semantic similarity between related words the way modern embeddings can.

11 Experimental Setup

Python Version	3.13.14
Scikit-Learn Version	(as installed in the local environment)
IDE	Jupyter Notebook
Operating System	Windows
Hardware	Personal laptop (CPU only)

12 Hyperparameter Selection

Parameter	Value
<code>train_test_split(test_size)</code>	0.2 (80:20 split) across all datasets
<code>train_test_split(random_state)</code>	42 (fixed, for reproducibility)
<code>DecisionTreeClassifier</code>	default Scikit-learn hyperparameters (Gini impurity, no max)
<code>LogisticRegression(max_iter)</code>	1,000 (Loan) / 100,000 (Diabetes, Spam) to ensure convergence
<code>KNeighborsClassifier(n_neighbors)</code>	3
<code>SelectKBest(k)</code>	1 (Iris, Loan, Diabetes) / 10 (Spam) / 300 (MNIST)

13 Results Summary

Dataset	Best Model	Best Accuracy
Iris Dataset	Decision Tree / KNN (tied)	1.000
Loan Amount Prediction	Decision Tree	0.974
Predicting Diabetes	Logistic Regression*	0.685*
Email Spam Classification	Decision Tree	0.967
MNIST Digit Recognition	Decision Tree	0.794

*As discussed in Section 8.5, this accuracy is misleading due to class imbalance – Logistic Regression predicted only the majority class, so precision/recall on the diabetic class are both 0.

14 Discussion

Across the five datasets, tree-based models (Decision Tree) were consistently competitive or better than Logistic Regression, particularly on datasets with non-linear or threshold-driven decision boundaries (Loan, Diabetes). Logistic Regression performed best when

either the classes were linearly separable in a simple feature space (not the case here) or after proper scaling (Diabetes), though it struggled badly with class imbalance. Feature selection consistently identified interpretable, domain-sensible top features – Petal-LengthCm for Iris, cibil_score for Loan, and spam-vocabulary words for the email dataset – which builds confidence that the pipelines are working correctly rather than overfitting to noise. The MNIST experiment additionally demonstrated that aggressive dimensionality reduction (784 to 300 features via Chi-square) can retain almost all predictive accuracy, which is valuable for computational efficiency. The main limitation observed across the notebooks is that default hyperparameters were used throughout (no cross-validation or grid search), and class imbalance (Diabetes, Spam) was not explicitly corrected via resampling or class weights, both of which would likely improve reported minority-class performance.

15 Conclusion

This experiment successfully demonstrated the end-to-end machine learning workflow – data loading, EDA, preprocessing, feature selection, model training and evaluation – across five different types of ML problems: multiclass tabular classification (Iris), binary tabular classification (Loan, Diabetes), binary text classification (Spam) and multiclass image classification (MNIST). All five problems were correctly identified as supervised classification tasks. The exercise reinforced how the same core Scikit-learn API (`train_test_split`, `SelectKBest`, `fit/predict`) generalises across very different data modalities, while also surfacing dataset-specific issues such as class imbalance (Diabetes, Spam) and multicollinearity (Iris) that must be accounted for when interpreting model performance.

16 References

- [1] Scikit-learn developers, "scikit-learn: Machine Learning in Python," 2024. [Online]. Available: <https://scikit-learn.org/stable/>
- [2] pandas development team, "pandas documentation," 2024. [Online]. Available: <https://pandas.pydata.org/docs/>
- [3] UCI Machine Learning Repository, "Iris Data Set." [Online]. Available: <https://archive.ics.uci.edu/dataset/53/iris>
- [4] Kaggle, "Loan Approval Prediction Dataset." [Online]. Available: <https://www.kaggle.com/>
- [5] Kaggle, "SMS Spam Collection Dataset." [Online]. Available: <https://www.kaggle.com/>
- [6] Y. LeCun, C. Cortes, and C. J. C. Burges, "The MNIST Database of Handwritten Digits." [Online]. Available: <http://yann.lecun.com/exdb/mnist/>